BS - KU5

→ Speicherverwaltung

○ Speicherhierarchie: → Speicher eines PCs setzt sich aus mehreren hierarchisch angeordneten Speichern zusammen

→ Aus sich der CPU erfolgt die Anordnung geordnet nach: → sinkender Zugriffsgeschwindigkeit

→ sinkenden Kosten

→ steigende Kapazität & Zugriffszeit

→ Maßeinheiten für Speicher:

1 KiB = 1024 Byte = 2^{10} B

1 MiB = 1024 KiB = 2^{20} B

1 GiB = 1024 MiB = 2^{30} B

→ Arten: ○ RAM = Random Access Memory

↳ flüchtiger Speicher (volatile)

○ ROM = Read Only Memory

↳ persistenter Speicher

○ SRAM = Static RAM → Caches

↳ sehr schnell, geringer Stromverbrauch, teuer, hoher Platzverbrauch

○ DRAM = Dynamic RAM → Hauptspeicher

↳ langsamer, hoher Stromverbrauch, günstiger & kleiner

BS-KUB

- **Speicherverwaltung**
  Teil des BS, welcher den Hauptspeicher verwaltet
  ↳ Aufgaben:
  → Verwaltung von freien und belegten Speicherbereichen
  → Zuweisung von Speicherbereichen an parallel ablaufende Prozesse zur Laufzeit
  → Freigabe nach Benutzung oder bei Prozessende
  → Auslagerung von Prozessen zwischen Hauptspeicher und Swap-Bereich im persistenten Speicher

- **Cache**
  Schneller Zwischenspeicher, in dem häufig benötigte Daten vorübergehend gespeichert werden, damit schnell auf sie zugegriffen werden kann.

  ↳ Cache zwischen Hauptspeicher und CPU

  ⇝ CPU will Daten lesen
  - Cache prüft, ob Daten vorhanden → Ja → So werden Daten geliefert (schnell)
    ↳ Wenn so wird Cache-Line, der die Daten enthält aus Hauptspeicher geladen, in den Cache gebracht & geforderte Daten an CPU geliefert.
    ↳ Falls voll, Verdrängungsstrategien bestimmt

  ⇝ CPU will Daten schreiben
  - Daten werden im Cache gespeichert.
    → Sofort in den Hauptspeicher geschrieben oder
    → erst dann in den Hauptspeicher geschrieben, wenn entsprechender Block verdrängt wird

BS - KÜB

→ CPUs arbeiten mit mehrstufigen Caches

→ Relokation: → Ein Prozess kann zu unterschiedlichen Zeiten unterschiedliche Speicherbereiche belegen.

- Physikalischer und virtueller Speicher
  → Der physikalische Speicher eines Computers ist der tatsächlich im Gerät verbaute Speicher.
  → Der virtuelle Speicher eines Prozesses ist der Speicher, der einem Prozess durch das BS zur Verfügung gestellt wurde.

- Adressen (physikalisch, virtuell)
  → Physikalisch (absolut): Referenz auf eine konkrete Stelle im physischen Hauptspeicher
  → Virtuell (logisch): Referenz auf Stelle innerhalb des virtuellen Speichers eines Prozesses unabhängig von aktueller Zuteilung von physikalischen Speicher

- Herausforderung der Speicherverwaltung
  Die Umsetzung virtueller Adressen in physikalische Adressen muss sehr effizient sein

↳ MMU

BS - KVB

→ Variante 1: Statische Partitionierung

- Unterteilung des Speichers in Partitionen fester Größe
- BS belegt eine Partition
- jedem Prozess wird fest eine Partition zugeordnet
- Nachteil: (-) Max. Anzahl aktiver Prozesse beschränkt
  (-) Partition wird nicht ganz genutzt
- Vorteile: (+) Adressumsetzung einfach

→ Schutz:

↳ Ein Prozess kann Speicherbereiche anderer Prozesse lesen/schreiben

→ Schutz: Unterstützung durch HW:

→ Basisregister: Anfangsadresse der Partition

→ Grenzregister: Ende des Speicherbereichs des Prozesses

- **Fragmentierung:**
  Zerstückelung eines Speichers in Bereiche, die nicht genutzt werden können.

- Interne Fragmentierung:
  Ungenutzter freier Speicher innerhalb von Speicherbereichen

- Externe Fragmentierung:
  Ungenutzter freier Speicher zwischen Speicherbereichen

BS - HÜB

→ Variante 2: Dynamische Partitionierung:

- Anzahl, Anfangsadresse und Größe der Partitionen und damit der eingelagerten Prozesse ändern sich dynamisch
- Vorteil: Exakte Zuweisung des angeforderten Speichers
- Nachteil: - Genauer Speicherbedarf eines Prozesses unbekannt
  - Mehr Verwaltungsarbeit
  - externe Fragmentierung
- Behandlung von Speicheranforderung zur Laufzeit:
  - → Belegen ungenutzter Lücke, falls vorhanden
  - → Verschiebung von anderen Prozessen, falls möglich
  - → Raum zum Wachsen: Bei der ersten Speicherbelegung zusätzlich Speicherplatz allozieren

:(sehr kompliziert

→ Variante 2a: Paging

- Speicherbereich eines Prozesses ist nicht zusammenhängend
  - → Hauptspeicher wird in gleich große Rahmen aufgeteilt // frames
  - → Jeder Prozess wird in gleich große Seiten aufgeteilt // pages
  - → Seitengröße = Rahmengröße
  - → Laden des Prozesses: Seiten werden in Rahmen geladen
- Vorteil: Keine Externe & nur geringe interne Fragmentierung
- Nachteil: Adressumrechnung aufwendig
  - ↳ Hilfe durch HW Memory Management Unit (MMU)

BS - VMB

↝ Quiz: 1 GiB physikalischer Speicher
Rahmengröße = 64 KiB.
Jeder Prozess 4 GiB virtueller Speicher

Rahmen insgesamt: $\frac{1\,GiB}{64\,KiB} = \frac{2^{30}\,B}{2^6 \cdot 2^{10}\,B} = 2^{14}\,B = 16 \cdot 2^{10} = 16384$

Seiten pro Prozess: $\frac{4\,GiB}{64\,KiB} = \frac{2^{32}}{2^{16}} = 2^{16} = 2^6 \cdot 2^{10} = 65536$

↝ Paging & Adressumrechnung

- Anzahl der Bytes einer Seite ist 2er Potenz (2^m), Anzahl Lage n
  - ↳ 1. Teil: n - m Bits
  - ↳ 2. Teil: m Bits
- Bedeutung einer virtuellen Adresse
  - ↳ n - m Bits: Seitennummern
  - ↳ m Bits: Offset des Bytes in der Seite, in der Byte steht
- Vorgehen
  - → Ist Anfangsadresse des Rahmens bekannt, in dem Seite abgespeichert ist, dann wird zu dieser Adresse nur das Offset addiert und so erhält man die physikalische Adresse
  - → Anfangsadresse des Rahmens: Rahmennummer n Bits nach links schieben

BS-KLB

- **Seitentabelle**

Eine Seitentabelle ist die Datenstruktur, die für das Umrechnen von virtuellen zu physischen Adressen verwendet wird. Es ist eine Liste von Rahmennummern.

↳ Pro Prozess eine Seitentabelle: # Einträge der Tabelle = # Seiten des Prozess
↳ Der i-te Eintrag enthält Rahmennummer der i-ten Seite
↳ Ein HW-Register enthält zu jedem Zeitpunkt Anfangsadresse der Seitentabelle des aktuellen Prozesses.
↳ Bei Prozesswechsel wird dieses HW-Register auf Anfangsadresse der Seitentabelle des neuen Prozesses gesetzt

- **Algorithmus:** Seitengröße 2^m Byte, Adressen der Länge n:
  ↳ Extrahiere Seitennummer (linke $n-m$ Bits)
  ↳ Verwende Seitennummer als Index in Seitentabelle, um Rahmennummer k zu bestimmen
  ↳ Physikalische Anfangsadresse des Rahmens: $k \times 2^m$
  ↳ Physikalische Adresse des Bytes: $k \times 2^m + \text{Offset}$
  ↳ Direkt Berechnet durch % Index

⇝ **Quiz:** Virtueller Adressraum: Größe 2^n Byte mit Seitengröße 2^u Byte.
Seitennummer aus höherwertigen X Bits der virtuellen Adresse und restliche Y Bit als Offset.

$X = n - u$, $Y = u$

BS-HÜB

→ Quiz: virtueller Adressraum 2^{16} B
physikalischer Speicher 2^{12} B
Rahmengröße 2^{8} B

	0	1	2	3	4
Seitentabelle	0100,	1101,	0010,	0000,	1111, 1001, ...

virtuelle Adresse	0000	0100	1110	1001
	↓		↓	
physikalische Adresse	1111		1110	1001

→ Variante 2b: Virtueller Speicher

- Es ist nicht notwendig, dass während Programmausführung alle Seiten im Hauptspeicher geladen sind.
- BS hält die gerade in Benutzung befindlichen Seiten eines Prozesses im Hauptspeicher und Rest auf Festplatte. → Virtueller Speicher
- Vorteile: (+) Prozess kann mehr Speicher zugeordnet bekommen, als physikalisch verfügbar
  (+) Physikalische Adressen können größer als virtuelle Adressen sein
  (+) Viele Prozesse können gleichzeitig Rahmen im Hauptspeicher belegen
- Vorgehen wie beim Paging
- Unterschiede:
  → Zu einem Zeitpunkt sind nicht alle Seiten eines Prozesses in Rahmen des Hauptspeichers abgelegt
  → Seiten, die nicht in einem Rahmen sind, heißen <u>ausgelagerte Seiten</u>
  → Zuordnung von Seiten zu Rahmen kann sich ändern

BS - MMU

↳ Virtueller Speicher: Adressberechnung
- wie beim Paging
- Present / Absent - Bit zeigt an ob Seite ausgelagert oder im Hauptspeicher

- Seitenfehler (Page Fault)
Ist beim Zugriff auf eine Seite diese in keinem der Rahmen im Hauptspeicher abgelegt, so erzeugt die MMU einen Interrupt. Diese Situation heißt page fault.

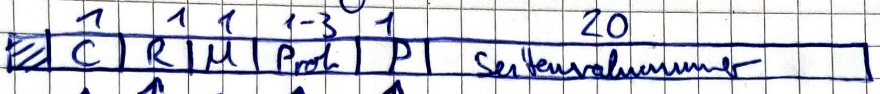
↳ Dabei wird fehlende Seite vom persistenten Speicher in freien Rahmen des Hauptspeichers eingelagert und Seitentabelle aktualisiert.

- Demand Paging
Die Einlagerung einer Seite als Reaktion auf Seitenfehler nennt man Demand Paging

↳ Ist kein Rahmen frei, so wird ein belegter Rahmen freigemacht
↳ Falls die zu ersetzende Seite während ihrer Zeit im Speicher
- verändert wurde (Dirty Page), so wird sie zuerst zurückgeschrieben. Der belegte Rahmen ist erst danach frei.
- nicht verändert wurde, ist kein Zurückschreiben erforderlich

BS-KUB

→ Seitentabelleneintrag:



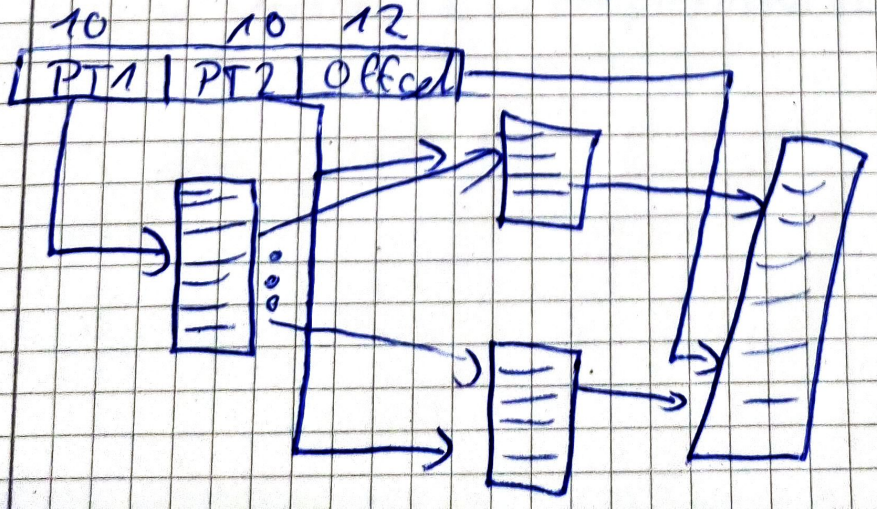
- Present / Absent - Bit: Seite im Hauptspeicher oder nicht
- Zugriffsrechte (Read, Write, Execute)
- Modified - Bit: Seite wurde beschrieben
- Referenced - Bit: Benutzt von Seitenersetzungsverfahren
- Cacheable - Bit: Erlaubt Herstellen des Caching

→ Umsetzung von Seitentabellen:

- Seitentabellen sind sehr groß ↯
  - ↳ Seitentabellen liegen im virtuellen Speicher
  - ↳ Der Teil der Seitentabelle, der den Eintrag der momentan aktuellen Seite enthält muss geladen sein!

→ Mehrstufige Seitentabellen

- Vorteil: Nicht alle Einträge einer Seitentabelle müssen existieren
- Einträge in Seitentabellen sind Rahmennummern, keine physikalischen Adressen
- Ablauf: → PT1 ist Offset in Seitentabelle erster Stufe, auf den 20 Bit Wert an dieser Stelle wird PT2 addiert
  → Um 12 Nullen ergänzt ist dies die Adresse eines Eintrages in der Seitentabelle der 2-Stufe.

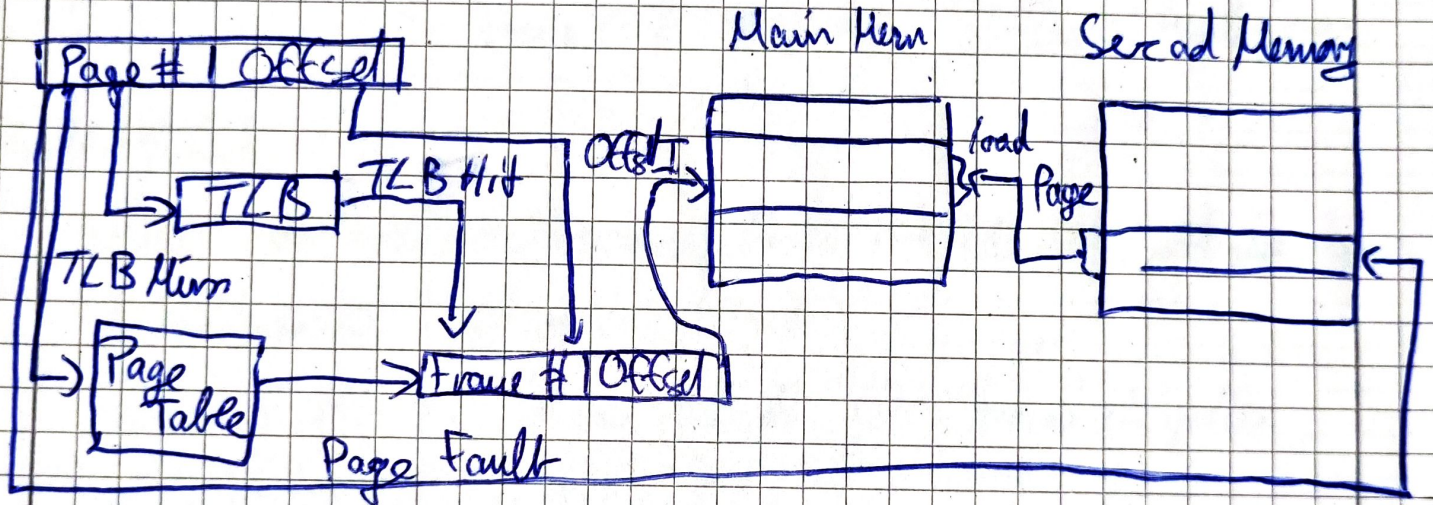


BS - HVB

- Translation Lookaside Buffer (TLB)

  Der TLB ist ein Cache mit Zuordnung von Seitennummern zu Rahmennummern

↳ TLB-Eintrag beschreibt eine Seite des virtuellen Adressraums
↳ Gespeicherte Information entspricht Eintrag in Seitentabelle
↳ Voll assoziativer Zugriff: HW kann Einträge parallel abfragen, um Vorhandensein eines Eintrages zu testen.



→ Bei TLB Miss wird ein Eintrag durch Eintrag in Seitentabelle ersetzt
⇒ Page Fault wird Interrupt erzeugt und Seite geladen, danach wird letzter Befehl wiederholt, d.h. Anfrage geht nochmals an TLB
⇒ Ersetzungsstrategien
  → am längsten nicht benutzte Eintrag
  → Falls Modified-Bit gesetzt ist, wird entsprechendes Modified-Bit in Seitentabelle im Hauptspeicher gesetzt

BS - MMB

⇝ Ablauf: Page Lookups

- Virtuelle Adresse im TLB
  Dauer: 1 TLB Zugriff + 1 Speicherzugriff
- Virtuelle Adresse nicht im TLB aber im Hauptspeicher
  Dauer: 1 TLB Zugriff + 1 Seitentabellenzugriff +
  1 Speicherzugriff + TLB aktualisieren
- Virtuelle Adresse nicht im TLB und nicht im Hauptspeicher
  Dauer: 1 TLB Zugriff + 1 Seitentabellenzugriff + 1 Speicherzugriff
  + 1 + Zugriff auf Festplatte + TLB aktualisieren

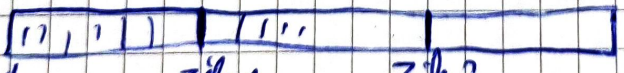
⇝ Trefferrate

- Anteil der Speicherreferenzen, die mittels TLB aufgelöst werden können, heißt Trefferrate (Hitrate)
  ↳ Hohe Trefferrate ist nur möglich, falls Prozess ein gutes Lokalitätsverhalten aufweist.

⇝ Rate der Seitenfehler

- E/A-Zeit & Speicherzugriffscharakteristika speziell für größere Seiten
  ↳ Lokalitätsprinzip!
- Lokalitätsprinzip
  Bei der Programmausführung konzentrieren sich Speicherzugriffe für Befehle/Daten auf enge Speicherbereiche

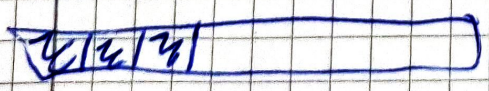
- eg!




```
int [][] a = new int [1000][1000];
for (int i = 0; i < 1000; i++)
  for (int j = 0; j < 1000; j++)
    a[i][j] = i * j;
```

```
int [][] a = new int [1000][1000]
for (int j = 0; j < 1000; j++)
  for (int i = 0; i < 1000; i++)
    a[i][j] = i * j
```

Lagsaus



BS - VVB

- **Working Set (WS)**
  Die Menge der Seiten, die ein Prozess innerhalb eines Zeitintervalls nutzt, nennt man Working Set.

↳ Kann zugeordneter Hauptspeicher WS nicht aufnehmen, so entstehen viele Seitenfehler.

↳ Thrashing:
Wird nach wenigen Zugriffen eine gerade verdrängte Seite wieder benötigt, so sinkt Ausführungsgeschwindigkeit auf einige Instruktionen je Seitenein-/ausgang

→ Einfluss Seitengröße
kleine Seite: viele Rahmen, wenig Seitenfehler, große Seitentabelle
große Seite: kleine Seitenfehler

→ Quiz:
BS mit virtuellem Speicher → Ø Trefferrate 35%
Einlagern einer Seite in Hauptspeicher 100 ms
Zugriff auf eine in Hauptspeicher befindliche Seite 10 ms

0,35 · 10 ms + 0,65 · (100 ms

= 68,5 ms

BS - HÜ3

→ Strategien für virtuellen Speicher

- Abrufstrategie (Fetch Policy)

→ Wann wird Seite in den Hauptspeicher geladen?

- Demand Paging: Bei Seitenfehler
- Prepaging: Einlagern der Seite, obwohl entsprechender Seitenfehler nicht vorliegt
  - → Sinnvoll zu Prozessbeginn
  - → Efficient, falls einzulagernde Seite auf Festplatte nebeneinander
- Austauschstrategie (Replacement Policy)?

→ Least-Recently-Used (LRU)

- Es wird die am längsten unbenutzte Seite ersetzt
- LRU arbeitet sehr gut, aber Implementierung aufwendig
- Effiziente Implementierung nur mit HW möglich
- Naive Implementierung:

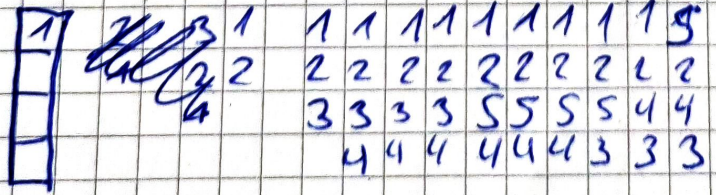
→ Verwalte Menge der aktuell eingelagerten Seiten in verketteter Liste

→ Zuletzt benutzte Seite steht vorn, die am längsten nicht benutzte Seite hinten

→ Bei Speicherzugriff wird zugegriffene Seite in Liste gesucht und an Kopf gesetzt

→ Wähle Seite am Ende der Liste als zu ersetzende Seite

eg! 1 2 3 4 1 2 5 1 2 3 4 5



BS - UVB

→ Clock-Algorithmus

- Rahmenverwaltung in zyklischer Liste in Form einer Uhr
- Jeder Rahmen hat Use-Bit, Uhrzeiger zeigt auf einen Rahmen
- Allg.: → Erstmaliges Einlagern: Use-Bit auf 1 setzen, Zeiger verschieben
  → Bei allen späteren Referenzierung wird Use-Bit auf 1 gesetzt
  → Bei Seitenfehler wird Rahmen überprüft, auf den Uhrzeiger zeigt.
  - Use-Bit = 0: Enthaltene Seite wird ausgelagert, die einzulagernde Seite in diesen Rahmen eingefügt und Uhrzeiger auf nächsten Rahmen vorgeschoben.
  - Use-Bit = 1: Es wird auf 0 gesetzt und Uhrzeiger eine Position vorgeschoben
  - Wiederhole bis Rahmen mit Use-Bit = 0 gefunden

BS - KÜB

→ NRU: Not Recently Used

- Verwendet der Status-Bits
  - R-Bit (Referenced): Wird gesetzt, wenn auf Seite zugegriffen wird
  - M-Bit (Modified): Wird gesetzt, wenn auf Seite schreibend zugegriffen wird
- Status-Bits werden durch HW aktualisiert
- Bei Prozesswechsel werden beide Status-Bits für alle Seiten des Prozesses zurückgesetzt
- Periodisch wird Referenced-Bit aller Seiten durch BS zurückgesetzt
- Bei Seitenfehler werden eingelagerte Seiten in Klassen eingeteilt:
  - → Klasse 0: nicht referenziert, nicht modifiziert, (R=0, M=0)
  - → Klasse 1: (R=0, M=1)
  - → Klasse 2: (R=1, M=0)
  - → Klasse 3: (R=1, M=1)
- Wähle zufällig aus der niedrigsten nichtleeren Klasse eine Seite zur Ersetzung aus
- NRU: Einfach, Effizient zu implementieren, nicht optimal aber akzeptable Performance

BS - HUB

~> Page Locking
- Rahmen können vom Ersetzungsverfahren ausgeschlossen werden
  ↳ DMA, LS genauso wie Speicher

~> lokale Seitenersetzung:
- Auswahl der zu ersetzenden Seite berücksichtigt nur Seiten des Prozesses selbst
- jeder Prozess bekommt feste Anzahl von Rahmen zugeordnet

~> Globale Seitenersetzung
- Alle sich im Speicher befindlichen Seiten, also auch die anderer Prozesse, werden in Betracht gezogen
- Rahmen werden dynamisch alloziert, d.h. Anzahl der Rahmen eines Prozesses ändert sich in Abhängigkeit der Gesamtlast.
- Seitenfehlerfrequenz

Seitenfehlerrate zwischen A und B wird als akzeptabel angesehen
-> Steigt über A, werden zusätzliche Rahmen zugeordnet
-> Sinkt unter B, werden Rahmen entzogen.

BS-UHB-Kap 4

(4-0)

a) Speicherhierarchie?

Speicher eines PCs setzt sich aus mehreren Ebenen mit unterschiedlichen Speichern zusammen.

Aus sicht der CPU erfolgt die Anordnung gerichtet nach
→ sinkender Zugriffsgeschwindigkeit & Kosten
→ steigender Speicherkapazität & Zugriffszeit.

b) Was ist der Unterschied zw. Interner & Externer Fragmentierung?

Interne Fragmentierung:
Ungenutzter freier Speicher innerhalb von Speicherbereichen

Externe Fragmentierung:
Ungenutzter freier Speicher zwischen Speicherbereichen

c) Virtuelle Speicher?

Vorteile: (+) Prozesse können mehr Speicher zugeordnet bekommen als physikalisch verfügbar
(+) Physikalische Adressen können größer als virtuelle Adressen sein
(+) Viele Prozesse können gleichzeitig im Hauptspeicher sein

Nachteil: (-) langsam → HW-Unterstützung

BS - HA3 - Ex 4

(4-1)

a) TLB?

Der TLB ist ein Cache mit Zuordng. von Seiten# zu Rahmen#.

c) Lokalitätsprinzip?

Lokalitätsprinzip:

-> Im Verlauf der Programmausführung konzentrieren sich Speicher Zugriffe für Befehle/Daten auf enge Speicherbereiche

(4-2)

auto a = new int[100][100];

P1:

```
for(int i=0; i<100; i++)
    for(int j=0; j<100; j++)
        a[i][j] = i*j;
```

P2:

```
for(int j=0; j<100; j++)
    for(int i=0; i<100; i++)
        a[i][j] = i*j;
```

↳ Lokalitätsprinzip verletzt

↳ mehr Page Faults

BS-KVB - Ex 4

(4-3) Speicherzugriff

In Cache → 15 ns um Wort zu lesen

Nicht Cache sondern Hauptspeicher → 45 ns um in Cache zu laden

Auf Festplatte → 10 ms um in Hauptspeicher/Cache.

Trefferquote Cache 80%, Trefferquote Hauptspeicher 70%

$$T = 0{,}8 \cdot 15\,ns + 0{,}2 \cdot (0{,}7 \cdot (45\,ns + 15\,ns) + 0{,}3 \cdot (10\,ms + 45\,ns + 15\,ns))$$

$$= 600{,}025\,\mu s \approx 600\,024\,ns$$

BS - HÜ3 - 6x4

(4-4) Virtueller Speicher

8 Bit große virtuelle Adresse.

3 Bit Seitennummer, Prozess verfügt über 6 Seiten

Seiten #	Rahmen #	P-Bit	M-Bit
0	4	1	1
1	7	1	0
2	1	0	0
3	0	1	1
4	2	0	0
5	1	0	0

1	0
2	1
4	2
8	3
16	4
32	5
64	6
128	7

→ Seiten / Rahmengröße:

3 Bit für Seitennummer ⇝ 8 - 3 = 5 Bit Offset

$2^5 = 32$ B

→ virtuelle Adresse: 53 = 001 1 0101

↳ Rahmen # = 7 = 111

physikalische Adr. 111 0 1 0 1 0 1 Kein Seitenfehler

→ virtuelle Adresse: 84 = 010 1 0100

↳ Rah # = 1 ⇝ Page Fault

001 10100

BS - KVB - Ex 4

(4-5) Zweistufige Seitentabelle

32-Bit System. 9-Bit für erste Stufe
10-Bit für zweite Stufe

virtuelle Adressraum = 4 GiB, physischer Adressraum = 1 GiB

→ Welchen Aufbau hat ein Eintrag in den Tabellen der zweiten Stufe & wie viele Bits werden für die Rahmen# benötigt?

32-Bit − 9 Bit = 23 Bit Einträge Erste Stufe
23-Bit − 10-Bit = 13 Bit Einträge Zweite Stufe

2^{13} = 8 KiB ≙ Seitengröße

$\frac{1\ GiB}{8\ KiB} = \frac{2^{30}}{2^{13}} = 2^{17}$ Seiten. ⇒ Rahmen# 17 Bit Breit

Eintrag in Tabelle | P | M | Reelle | Rahmen#

→ Wie groß ist eine Seite & wie viele Tabellen muss eine zweistufige Seitentabelle mind. besitzen, wenn ein Prozess mit 23 MiB verwaltet werden soll?

Seitengröße = 8 MiB

$\frac{23\ MiB}{8\ MiB} = \frac{23 \cdot 2^{20}}{2^{13}} = 23 \cdot 2^{7} = 2944$

~~13 Bit für Eintrag in Zweite Stufe — 2 Tabellen~~
Erste Stufe Zweite Stufe

Einträge einer Tabelle 2. Stufe = 2^{10} = 1024
↳ 1 Tabelle à 1. + 3 · Tabelle 2. = 4 Tabellen

BS-ÜB-Ex 4

(4-6) Second-chance algorithm

Prozess vier Rahmen

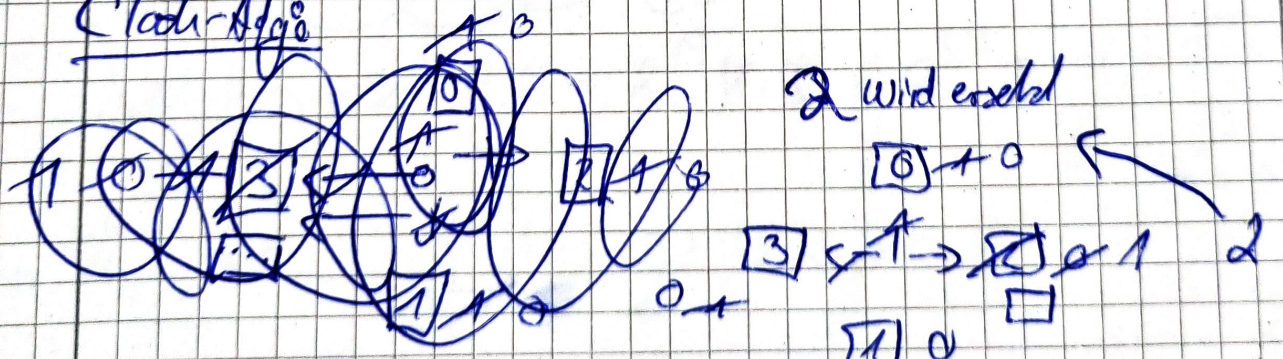
Seite #	Rahm #	Einfügt laden	Zeitpunkt letzte Zugr.	R-Bit	M-Bit
0	2	26	162	1	0
1	1	130	160	0	0
2	0	60	161	0	1
3	3	20	163	1	1

Page Fault ↯

LRU: ⇒ Ersetze Seite 1!
↳ 160 < 161 < 162 < 163

URU: Ersetze Seite 1!
Since Klasse 0: R=0, M=0

Clock-Algo


0
1
2
3
0
1
2
3
0
1
2
3

BS-Ü/B-Tasche1

1.1

20 Bit Speicheradressen, adressiert werden Bytes
Wie groß ist der adressierbare Speicher in MiB

$2^{20}\,B = 2^{10}\,MiB = 1024\,MiB$

1.2 32 Bit Speicheradressen, adressiert wird Bytes
Wie groß ist der adressierbare Speicher in MiB?

$2^{32} = 2^{20+2}\,MiB = 2^{10+2}\,MiB = 2^2\,GiB = 4\,GiB$

1.3 Welche Adressbreite benötigt ein Prozessor mindestens, wenn bei Wortadressierung (Wortlänge 2 B) ein Speicher der Größe 512 MiB ~~B~~ verwaltet werden soll?

~~$2^{\quad} B = 2^{\quad} \quad l = 8 \rightarrow 8$~~

$512\,MiB = 2^9\,MiB = 2^{19}\,MiB = 2^{29}\,B$

$\frac{2^{29}}{2}\,B = 2^{28}$ Wörter $\rightarrow$ 28 Bit.

Size →

BS-KUB Zettel 1

(1-0)

Processor hat 64 Register und einen Befehlssatz von 12 Befehlen.
Instruktion besteht aus einem Opcode und der Angabe eines Registers.

a) Wie viele Bits benötigt eine Instruktion?

Opcode : 4 Bit
Regis : 6 Bit
} 10 Bit

1	0
2	1
4	2
8	3
16	4
32	
64	

b) Wie viel Speicher in Bytes belegt ein Programm mit 1000 Anweisungen, falls die Instruktion byte-aligned abgelegt werden?

10 Bit in 2 Byte ⇝ 2000 Byte

(1-3) Opcode (k bit) & Adresse (n bit)

Der Befehlssatz des Processors soll verdoppelt werden, wobei die Gesamtlänge der Maschinenbefehle.

$k + n \Rightarrow (k+1) + (n-1)$

↓ Befehlssatz verdoppelt

↓ Adressierbare Adressen halbiert.

BS - HÜB - Zusatz 2

Virtueller Speicher:

32 - Bit virtuelle Adressen & 2-Stufige Seitentabellen.
Tabellen der 1. Stufe werden 8 Bit
Tabellen der 2. Stufe werden 11 Bit der virtuellen Adresse verwendet
Hauptspeicher 1 GiB
Die Teile der Seitentabellen werden immer jeweils komplett in ein Stück abgelegt.
Die Größe eines Eintrages in Seitentabelle beträgt 4 B.

a) Wie groß ist eine Speicherseite?

$32 - 8 - 11 = 13$ Bit $\Rightarrow 2^{13}$ B $=$ **8 KiB** $\hat{=}$ Seitengröße

b) Aus wie viele Seiten besteht der virtuelle Adressraum eines Prozesses?

virtueller Adressraum $= 2^{32}$ B $= 4$ GiB $\hat{=} 2^{\text{Länge}}$!

$$\frac{4\ \text{GiB}}{8\ \text{KiB}} = \frac{2^{32}}{2^{13}} = 2^{19} = \underline{524\,288}$$

c) Wie viel Speicher belegt maximal die Seitentabelle?

1. Tabelle der 1. Stufe und $2^8 = 256$ Tabellen der Stufe 2
↳ 257 Tabellen ⇝ 257 * 8 KiB = **2056 KiB**

BS-UVB- Zusatz 2

(2-4)

Wie viele Bits stehen für Statusinformation in der Seitentabelle zur Verfügung?

$$\#\text{Rahmen} = \frac{1\,\text{GiB}}{8\,\text{KiB}} = \frac{2^{30}}{2^{13}} = 2^{17} \rightsquigarrow 17\,\text{Bit zur Darstellung}$$

$$4\,\text{B} = 4 \cdot 8\,\text{Bit} = 32\,\text{Bit}$$

$$32\,\text{Bit} - 17\,\text{Bit} = \underline{15\,\text{Bit}}$$

(2-8) Wie viele Hauptspeicherzugriffe werden benötigt um ein Byte, das von einer virtuellen Adresse adressiert wird zu lesen?

(Nicht im Datencache vorhanden & alle notwendigen Seiten & Tabelleneinträge liegen im Hauptspeicher)

$$1\,(\text{Erste Tabelle}) + 1\,(\text{zweite Tabelle}) + 1\,(\text{Daten}) = 3$$

BS - KVB - Praxis 4

-> ipcs -s zeigt existierende Semaphore an.

-> semOpg -> Puffer für Semaphore

-> int semop(int semid, struct sembuf* sops, size_t nsops)
↳ führt die eigentliche operation auf dem Semaphore aus.
sops ist ein Feld von Operationen & nops ist die # der Operationen

Thread sichere Queue:

```
void queue_init(queue_t *queue) {
    [...]                                   -> member von Queue
    pthread_mutex_init(&queue->Mutex, Null)
    pthread_cond_init(&queue->canWrite, Null)
    pthread_cond_init(&queue->canRead, Null)
}

void queue_put(queue_t * queue, char b){
    pthread_mutex_lock(&queue->mutex);
    while(queue->size >= QueueMax){
        pthread_cond_wait(&queue->canWrite, &queue->mutex);
    }
    // FIFO stuff
    q.ua->size++;
    pthread_cond_signal(&queue->canRead);
}
```

cond

mutex